R pour l'archéologue

Exploration, visualisation et analyse de données

Nicolas Frerebeau

2022-02-09

Université Bordeaux Montaigne
Département d'Histoire de l'Art et d'Archéologie

Nicolas Frerebeau

MSHS-Toulouse (UAR 3414)
Maison de la Recherche
Université Fédérale de Toulouse – Jean Jaurès
5 allées Antonio Machado
F-31058 Toulouse cedex 9

Archéosciences Bordeaux (UMR 6034)
Maison de l'Archéologie
Université Bordeaux Montaigne
F-33607 Pessac cedex

Sommaire

Avant propos iii
Licence . . . iii
Contributions . . . iv
Conventions d'écriture . . . iv
Reproductibilité . . . iv

Introduction 1
Pourquoi travailler avec R ? . . . 1
Ressources complémentaires . . . 2

Installation 5
Installer R . . . 5
Installer RStudio . . . 5

Premiers pas avec R 9

1. Installation 11

2. Environnement de travail 13

3. Structure des données 15
3.1. Type de variables . . . 15
3.2. Tableaux de données . . . 15

Éléments de statistiques 17

4. Statistique descriptive 19

5. Relations entre variables 21

6. Visualisation de données 23

Chronologie 25

7. Remarques préliminaires 27

8. Chronologie qualitative 29

9. Chronologie quantitative 31

Cartographie 33

10. Coordonnées et projections 35

11. Format de données 37

Annexes 39

A. Bonnes pratiques 41
A.1. Travailler par projets 41

B. Style de programmation 43
B.1. Dénominations 43
B.2. Syntaxe 46
B.3. Commentaires 52

Bibliographie 53

Avant propos

Ce livre s'adresse aux personnes qui souhaitent analyser, visualiser et modéliser des données archéologiques à l'aide du langage R. Cet ouvrage vous permet d'acquérir les connaissances et les compétences nécessaires pour aborder un large éventail de problèmes liés aux données archéologiques.

Bien qu'il soit initialement conçu pour les étudiants du master mention *Archéologie, sciences pour l'archéologie* de l'Université Bordeaux Montaigne, ce livre s'adresse à tout étudiant, doctorant, post-doctorant ou archéologue professionnel souhaitant découvrir le langage R et s'y initier pour l'analyse de données archéologiques.

Ce livre s'adresse à des utilisateurs débutants, il ne demande aucun prérequis en programmation. Bien que des notions élémentaires soient rappelées au fil de l'ouvrage, ce livre n'est pas un manuel de statistiques. Le lecteur trouvera des suggestions de lecture complémentaires à la fin de l'introduction.

Licence

Ce livre est mis à disposition selon les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International[1].

Le code contenu dans ce livre est simultanément disponible sous la licence MIT[2].

Une entrée BibTeX pour citer ce document :

```
@Book{rarcheo,
  title = {R pour l'archéologue: Exploration, visualisation et analyse de données},
  author = {Nicolas Frerebeau},
  year = {2021},
  publisher = {Université Bordeaux Montaigne},
  address = {Pessac, France},
```

[1] Cela signifie que vous êtes libre de copier, distribuer et communiquer ce livre par tous moyens et sous tous formats, de modifier, transformer et créer à partir du matériel de ce livre. En contrepartie, vous devez créditer la source d'origine en précisant si vous l'avez modifiée, vous devez partager sous la même licence et vous ne pouvez pas en faire un usage commercial.

[2] Cela signifie que vous êtes libre de l'utiliser dans vos propres projets, à condition de citer la source.

```
  url = {https://rstats.archaeo.science},
}
```

Contributions

Le code source de ce livre est disponible sur GitHub. N'hésitez pas à y ouvrir un ticket pour signaler toutes les erreurs que vous pourrez constater ou proposer des modifications.

Conventions d'écriture

Par la suite, on écrira `foo()` pour désigner une fonction, `bar` pour désigner les variables et les paramètres des fonctions et `baz/` pour désigner les chemins. Les noms de *packages* sont indiqués en italique.

Les commentaires dans le code sont précédés par le signe `#`. Les blocs de code alternent l'entrée et la sortie. La sortie est commentée (précédée par `#>` pour la distinguer des commentaires standards) de sorte que vous pouvez facilement copier et coller les exemples dans R.

Reproductibilité

Ce livre a été écrit dans RStudio avec le package R *bookdown* (Xie, 2016). Le site web est hébergé par GitHub Pages et automatiquement mis à jour après chaque commit par GitHub Actions (cette version a été compilée le 09 Feb 2022).

Les données utilisées dans les exemples sont issues des packages *datasets* (R Core Team, 2021) et *folio* (Frerebeau, 2021). Les exemples reposant sur la génération de nombre aléatoires sont reproductibles en utilisant `set.seed(12345)`.

Cette version du livre a été réalisée avec R version 4.1.2 (2021-11-01), les packages utilisés sont résumés dans le tableau 1.

Tab. 1. : Liste des packages utilisés.

Package	Version	Source	Référence
bookdown	0.24	CRAN (R 4.1.2)	Xie (2021a)
bslib	0.3.1	CRAN (R 4.1.2)	Sievert et Cheng (2021)
desc	1.4.0	CRAN (R 4.1.2)	Csárdi et al. (2021)
DiagrammeR	1.0.6.1	CRAN (R 4.1.2)	Iannone (2020)
downlit	0.4.0	CRAN (R 4.1.2)	Wickham (2021)

Package	Version	Source	Référence
folio	1.0.0	CRAN (R 4.1.2)	Frerebeau (2021)
knitr	1.36	CRAN (R 4.1.2)	Xie (2021b)
markdown	1.1	CRAN (R 4.1.2)	Allaire et al. (2019)
mime	0.12	CRAN (R 4.1.2)	Xie (2021c)
rmarkdown	2.11	CRAN (R 4.1.2)	Allaire et al. (2021)
sessioninfo	1.2.1	CRAN (R 4.1.2)	Wickham et al. (2021)
webshot	0.5.2	CRAN (R 4.1.2)	Chang (2019)
xml2	1.3.2	CRAN (R 4.1.2)	Wickham et al. (2020)

Introduction

Pourquoi travailler avec R ?

R est libre et gratuit Le développement de R a débuté en 1991 à l'initiative de Ross Ihaka et Robert Gentleman au sein du département de statistiques de l'université d'Auckland (Nouvelle-Zélande). En 1995 la première version officielle est publiée et R devient un logiciel libre, distribué sous licence GNU. Le code source de l'ensemble du système R est ainsi accessible à tous, en garantissant quatre libertés essentielles :

- La liberté de faire fonctionner le programme comme vous voulez, pour n'importe quel usage (liberté 0) ;
- La liberté d'étudier le fonctionnement du programme, et de le modifier pour qu'il effectue vos tâches informatiques comme vous le souhaitez (liberté 1) ; l'accès au code source est une condition nécessaire ;
- La liberté de redistribuer des copies, donc d'aider les autres (liberté 2) ;
- La liberté de distribuer aux autres des copies de vos versions modifiées (liberté 3) ; en faisant cela, vous donnez à toute la communauté une possibilité de profiter de vos changements ; l'accès au code source est une condition nécessaire.

R est modulable Les libertés offertes par la licence GNU et la structure modulaire de R permettent à toute personne de développer des extensions (appelées *packages*) et de les mettre à disposition de la communauté des utilisateurs. De nombreux packages fournissent des interfaces entre le langage R et d'autres langages ou logiciels libres, ce qui en fait un outil polyvalent, une passerelle permettant d'utiliser de nombreuses méthodes et algorithmes spécialisés. Ces packages peuvent être publiés et installés depuis le *Comprehensive R Archive Network*[3] (CRAN) qui distribue également le code source de R et sa documentation. Fin 2021, plus de 22500 packages étaient disponibles sur le CRAN (fig. 1).

R facilite la reproductibilité R est un langage de programmation qui fonctionne à l'aide de scripts. Ces scripts permettent de définir une suite d'instructions à exécuter et contiennent ainsi l'ensemble des étapes de l'analyse. Il est possible de modifier ou de rééxecuter une analyse pour mettre à jour des résultats ou de réutiliser un même script avec des données différentes. Ces scripts peuvent être archivés,

[3] Il existe d'autres archives, parfois spécialisées comme Bioconductor pour la biologie.

partagés, soumis à l'examen des pairs et publiés au bénéfice d'une science ouverte, transparente et reproductible.

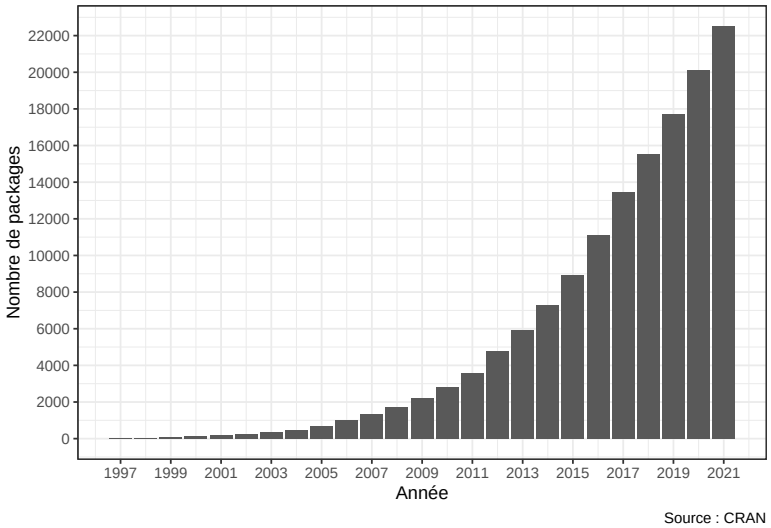


FIG. 1. : Évolution du nombre de packages disponibles sur le CRAN par année.

R s'est ainsi progressivement imposé comme un outil de référence pour l'analyse de données dans de nombreuses disciplines, notamment en archéologie (fig. 2). Si l'apprentissage du langage peut paraître difficile pour qui n'a jamais programmé, R bénéficie d'une communauté très active d'utilisateurs, de contributeurs et de développeurs. La documentation est abondante et il est toujours possible de trouver de l'aide en ligne. Si vous faites face à une difficulté, il fort probable quelqu'un se soit déjà posé la même question que vous et que la réponse se trouve sur Stack Overflow ou sur les listes de diffusions R-help et R-devel (actives depuis 1996).

Enfin, l'usage de R, comme de tout autre langage des programmation, conduit très souvent à généraliser un cas particulier. C'est ainsi un très bon moyen pour améliorer sa réflexion et, éventuellement, revoir la façon dont les problèmes sont posés.

Ressources complémentaires

En complément de la documentation officielle pour démarrer avec R et comprendre la structure des données, il existe de très nombreux ouvrages dont la plupart sont consultables en ligne. *R in a Nutshell* de ADLER (2010) et *R cookbook* de LONG (2019) couvrent les usages élémentaires du langage (plus qu'ils ne sont traités ici) et sont d'une aide précieuse pour un usage courant.

L'ouvrage de référence de MURRELL (2019), *R graphics*, offre une vue d'ensemble des fonctionnalités graphiques de R et de leur usage avancé. *R Graphics Cookbook* (CHANG,

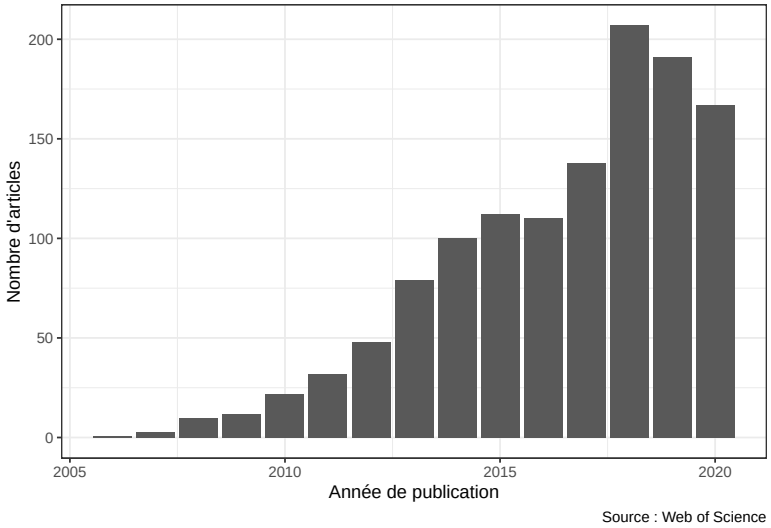


FIG. 2. : Nombre d'articles d'archéologie citant R publié chaque année (répliqué d'après SCHMIDT & MARWICK, 2020).

2012) constitue également une référence indispensable et fournit des recettes clé en main pour la création de graphiques. Quiconque s'attachant à la visualisation de données consultera avec intérêt *Fundamentals of Data Visualization* de WILKE (2019).

L'utilisateur confirmé trouvera dans les ouvrages *Advanced R* (WICKHAM, 2019) et *Efficient R programming* (GILLESPIE, 2016) de quoi approfondir sa compréhension du langage et développer sa pratique. *R packages* de WICKHAM (2015) est l'ouvrage idéal pour tout utilisateur souhaitant écrire son premier package et se former aux outils de développement (*devtools*, *roxygen2*, *testthat*...)[4].

Quelques ouvrages thématiques susceptibles d'intéresser l'archéologue[5] :

- *Geocomputation with R* (LOVELACE, 2020), un livre sur l'analyse, la visualisation et la modélisation des données géographiques.
- *Text Mining with R* (SILGE, 2017), un livre sur l'analyse textuelle avec R.

Enfin, on trouvera sur R-bloggers (un agrégateur de blogs) l'actualité du langage et de nombreux tutoriels.

[4]Voir également la documentation officielle pour le développement d'extensions de R, la description du processus de création des modules complémentaires, l'écriture de la documentation, les interfaces et l'API : *Writing R Extensions*.

[5]On trouvera sur le site du package bookdown et sur le site du Big Book of R de nombreux ouvrages supplémentaires, librement consultables en ligne. Le site du projet R propose également une liste d'ouvrages dédiés à R et ses applications.

Conseil

Le site collaboratif Rzine a pour objectif de favoriser la diffusion et le partage de connaissances sur la pratique de R en SHS :

> Rzine.fr s'efforce de dessiner les contours de la communauté R en SHS. Il donne aux débutant · e · s un accès simple à l'information et orienté vers une utilisation autonome. Il offre aux utilisateur · rice · s de niveau intermédiaire la possibilité d'étendre leurs pratiques et de s'ouvrir à d'autres méthodes parfois issues d'autres disciplines. Enfin, il s'agit d'un espace de diffusion pour les utilisateur · rice · s avancé · e · s, auteur · e · s de développements ou de documentation. — Rzine

Rzine recense de nombreuses ressources, fiches et manuels, *open source* et rédigés en français. Le site du projet permet également suivre l'actualité du langage et des évènements liés à la pratique de R en SHS.

Installation

Installer R

R est fonctionne sur les principaux systèmes d'exploitation. Vous pouvez télécharger les fichiers d'installation depuis le site du CRAN. Sous Windows et Mac OS, lancez l'exécutable après l'avoir téléchargé et suivez les étapes d'installation en laissant les options par défaut.

R est disponible dans les dépôts d'Ubuntu. Cependant, les paquets inclus dans les dépôts Ubuntu par défaut sont souvent un peu anciens. Il peut être préférable d'installer R à partir du dépôt du CRAN pour bénéficier des dernières versions.

Pour installer R sur Ubuntu :

```
sudo apt update
sudo apt -y install r-base r-base-dev
```

Important

Des changements importants peuvent subvenir entre les versions de R, entraînant parfois des problèmes de compatibilité. Il est fortement recommandé d'utiliser la dernière version mineure de R et de maintenir son installation à jour.

Installer RStudio

RStudio est un environnement de développement intégré (*interactive development environment*, IDE)[6]. RStudio *n'est pas* R, pas plus qu'il n'offre une interface graphique à R. Il s'agit d'un logiciel utilitaire qui permet d'éditer et d'exécuter votre code, ainsi que de réaliser des actions connexes telles que l'installation de packages ou l'import de données. RStudio dispose notamment d'un système de gestion de projets qui permet d'organiser son travail.

[6] RStudio n'est pas le seul IDE, des utilisateurs avancés peuvent lui préférer *Emacs Speaks Statistics* (ESS), un plugin pour l'éditeur Emacs.

RStudio Desktop est disponible sous licence *open-source* pour les principaux systèmes d'exploitation et peut être téléchargée depuis le site de la société éditrice.

Pour installer RStudio sur Ubuntu, ouvrez un terminal dans le dossier de téléchargement et lancez la commande suivante (le nom du fichier peut différer en fonction de la version) :

```
sudo dpkg -i rstudio-2021.09.2-382-amd64.deb
```

Important

Après l'installation de RStudio, la première étape consiste à en changer les réglages par défaut, pour éviter qu'il ne conserve des éléments entre deux sessions de travail. Ouvrez les options globales et modifiez les réglages suivants (fig. 3) :

- Décochez *Restore .RData into workspace at startup*.
- Réglez *Save workspace to .RData on exit* sur *never*.

Ces deux réglages vous épargneront de mauvaises surprises (et vous éviteront de faire débuter tous vos scripts par `rm(list = ls())`).

L'interface de RStudio se présente sous la forme d'un ensemble d'onglets répartis en quadrants (fig. 4). Par défaut, de haut en bas et de gauche à droite :

- Le premier quadrant contient les documents en cours d'édition (scripts…), permet d'explorer les données et offre un accès rapide à certaines commandes en fonction du contexte (exécuter un script, compiler un document…).
- Le second quadrant permet d'accéder aux objets présent dans l'environnement et de voir l'historique des commandes.
- Le troisième quadrant contient la console.
- Le quatrième quadrant permet de naviguer dans les fichiers, de visualiser les graphiques (générés depuis la console ou à l'aide d'un script), de gérer les packages et d'afficher l'aide.

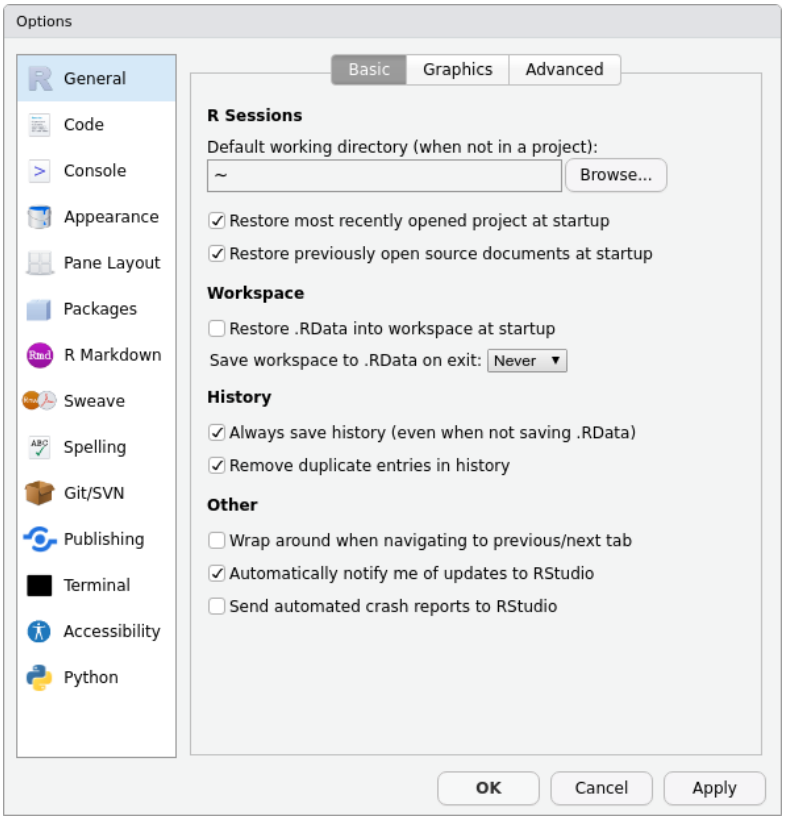


FIG. 3. : Configuration globale de RStudio Desktop.

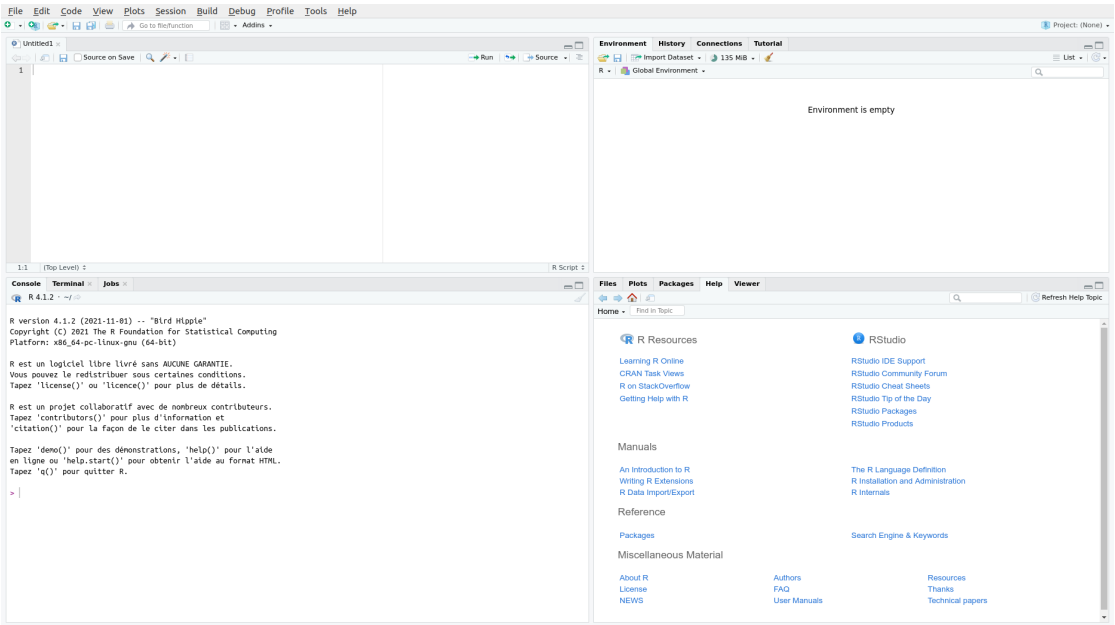


FIG. 4. : L'interface de RStudio Desktop.

Première partie

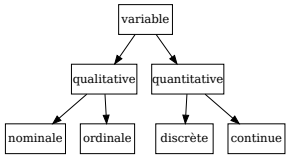
Premiers pas avec R

1. Installation

2. Environnement de travail

3. Structure des données

3.1. Type de variables



3.1.1. Variables quantitatives

3.1.2. Variables qualitatives

3.1.3. Transformations de variables

3.2. Tableaux de données

Deuxième partie

Éléments de statistiques

4. Statistique descriptive

5. Relations entre variables

6. Visualisation de données

Troisième partie

Chronologie

7. Remarques préliminaires

En archéologie, le processus de datation consiste au "placement dans le temps d'événements les uns par rapport aux autres ou par rapport à toute échelle établie de mesure temporelle" (Dean, 1978). Cela implique de distinguer les méthodes de datation dites *relatives* (qui fournissent une séquence chronologique ordonnée d'événements) et les méthodes de datation dites *absolues* (qui donnent une indication calendaire et peuvent fournir la durée d'un événement) (O'Brien & Lyman, 2002).

À proprement parler, il n'existe pas de datation absolue, étant donné la façon dont les dates sont produites et que toute date s'inscrit dans un référentiel particulier. La distinction entre temps absolu et temps relatif peut être reformulée plus clairement comme quantifiable ou non quantifiable (O'Brien & Lyman, 2002) : les dates absolues "sont exprimées en points sur des échelles standard de mesure du temps" (Dean, 1978).

Nous garderons ici la distinction entre une *date* et un *âge* telle que formulée par Colman et al. (1987) : "une date est un point précis dans le temps, alors qu'un âge est un intervalle de temps mesuré en arrière à partir du présent". Traiter des dates en archéologie peut être délicat si l'on ne tient pas compte des sources de l'information chronologique. Dans la plupart des cas, une date représente un *terminus* pour un assemblage archéologique donné. C'est-à-dire une date avant (*terminus ante quem*) ou après (*terminus post quem*) laquelle l'événement étudié a eu lieu. Enfin, on gardera à l'esprit qu'il n'existe pas de datation sans incertitude en archéologie (même si cette dernière ne peut pas toujours être quantifiée).

8. Chronologie qualitative

9. Chronologie quantitative

Quatrième partie

Cartographie

10. Coordonnées et projections

11. Format de données

Cinquième partie

Annexes

A. Bonnes pratiques

A.1. Travailler par projets

Une bonne pratique liée à l'utilisation de R consiste à organiser l'ensemble de son analyse en un projet, c'est à dire un dossier autonome contenant l'ensemble des fichiers nécessaires à l'analyse (dont les données brutes et le fichiers générés). Une telle organisation doit permettre de déplacer le dossier sur son ordinateur ou sur un autre ordinateur, tout en maintenant le bon fonctionnement du code (absence de liens brisés, de dépendances manquantes…).

Ce dossier, ou répertoire de travail (*working directory*), peut être défini ou retrouvé à l'aide des fonctions `setwd()` et `getwd()`, respectivement :

```
# Définition du répertoire de travail
setwd("/chemin/vers/mon/dossier")
```

RStudio dispose d'un mécanisme permettant de créer facilement de tels projets. Un projet est reconnaissable par la présence d'un fichier `.Rproj`. Ce fichier marque le dossier de plus haut niveau au sein d'un projet (répertoire de travail), à partir duquel des chemins d'accès relatifs peuvent être utilisés pour lire ou écrire des fichiers.

Il est possible de créer un projet dans RStudio à partir du menu déroulant situé en haut à droite de l'interface ou en cliquant sur *New Project…* depuis le menu *File*. Le projet peut alors être créé soit dans un nouveau dossier, soit en transformant un dossier existant (fig. A.1).

L'organisation des fichiers et des sous-dossiers au sein d'un projet relève des habitudes de travail de chacun. S'il n'existe pas de consensus sur la manière d'organiser un projet, il peut être avantageux de suivre certaines conventions, comme celles utilisées par le package *rrtools*.

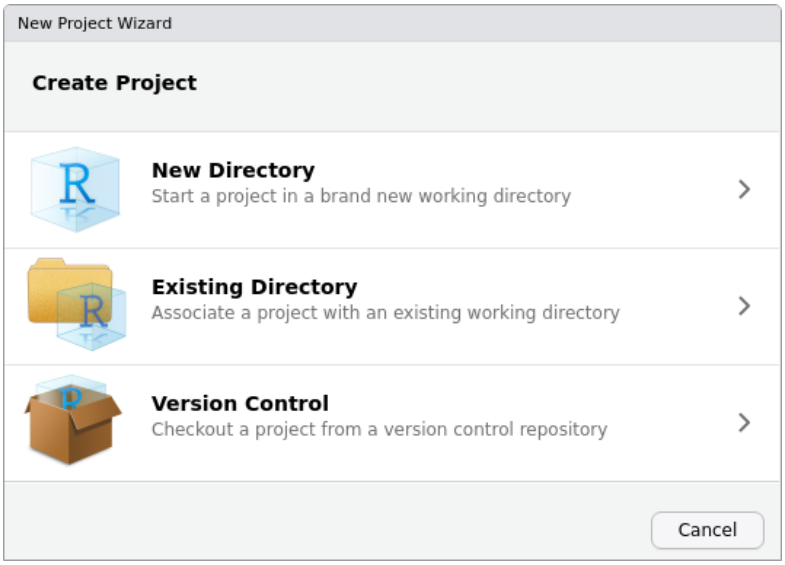


FIG. A.1. : Création d'un projet avec RStudio.

B. Style de programmation

Cette section propose un style de programmation destiné à accompagner les utilisatrices et utilisateurs débutants. Ce style regroupe des recommandations pour l'écriture du code et permet d'en faciliter la lecture et la compréhension. Ce style est dérivé du *tidyverse style guide* et du livre *rOpenSci Packages*.

B.1. Dénominations

En règle générale, les abréviations doivent être évitées lors de la dénomination de fichiers ou d'objets.

B.1.1. Nommer les fichiers

Les noms de fichiers doivent utiliser l'extension `.R`.

```
# GOOD
plot.R

# BAD
plot
```

Les noms de fichiers doivent être parlants.

```
# GOOD
plot.R

# BAD
Untitled1.R
```

Les noms de fichiers ne doivent pas contenir de `/` et d'espaces. À la place, utilisez un tiret (`-`) ou un trait de soulignement (`_`).

```
# GOOD
read_csv.R
plot-methods.R

# BAD
read csv.R
```

Les noms de fichiers doivent utiliser des lettres du latin de base, et non du supplément latin-1, et doivent être en minuscules.

```
# GOOD
plot.R

# BAD
Plot.R
données.R
```

Utilisez des verbes signifiants pour les noms de fichiers.

```
# GOOD
fit_model.R

# BAD
addition.R
```

Si le fichier ne contient qu'une seule fonction, nommez-le par le nom de la fonction.

B.1.2. Nommer les variables

Les noms des variables doivent être aussi courts que possible.

Les noms des variables doivent être des substantifs évocateurs.

Les noms de variables doivent être en minuscules.

Ne séparez jamais les mots à l'intérieur du nom par `.` (réservé pour les méthodes S3) et n'écrivez pas en PascalCase (réservé pour les définitions de classes S4). Utilisez plutôt un trait de soulignement (`_`).

```
# GOOD
std_dev <- 3

# BAD
std.dev <- 3
StdDev <- 3
```

N'utilisez pas les noms de fonctions et de variables existantes (en particulier celles qui sont propres à R).

```
# GOOD
std_dev <- 3

# BAD
T <- 1
c <- 2 * 2
mean <- 10
```

B.1.3. Nommer les fonctions

Les noms de fonction doivent contenir un verbe qui fait référence à l'action principale de la fonction.

Les noms de fonction doivent être en snake_case. N'utilisez `.` que pour le dispatching des méthodes S3.

Le schéma de nommage `object_verb()` doit être préféré aussi souvent que possible. Ce schéma est facile à lire et à compléter automatiquement.

```
# GOOD
peak_detect()

# BAD
addition()
readFile()
```

Évitez de réemployer des noms de fonctions déjà utilisés par les packages de base ou d'autres packages populaires.

B.1.4. Nommer les classes S4

Les noms des classes doivent être des substantifs en PascalCase avec une majuscule initiale et la première lettre de chaque mot concaténé en majuscule.

B.2. Syntaxe

B.2.1. Longueur de ligne

La longueur maximale des lignes est limitée à 80 caractères.

Ne mettez pas plus d'une déclaration par ligne. N'utilisez pas de point-virgule pour terminer la commande.

```
# GOOD
x <- 1
x <- x + 1

# BAD
x <- 1; x <- x + 1
```

B.2.2. Appel de fonction

Dans un appel de fonction, spécifiez les arguments par leur nom. Ne spécifiez jamais les arguments par un nom partiel et ne mélangez jamais la position et le nom complet.

```
# GOOD
mean(x, na.rm = TRUE)

# BAD
mean(x, na = TRUE)
```

Les arguments obligatoires doivent être placés en premier, suivis des arguments facultatifs.

```
# GOOD
raise_to_power(x, power = 2.7)

# BAD
raise_to_power(power = 2.7, x)
```

L'argument `...` doit se trouver soit au début, soit à la fin.

```
# GOOD
standardize(..., scale = TRUE, center = TRUE)

# BAD
standardize(scale = TRUE, ..., center = TRUE)
```

Définissez les arguments par défaut à l'intérieur de la fonction en utilisant l'idiome `NULL`, et évitez la dépendance entre les arguments.

Validez toujours les arguments dans une fonction.

B.2.3. Affectation

Utilisez `<-` au lieu de `=` pour l'affectation.

```
# GOOD
x <- 1

# BAD
x = 1
1 -> x
```

B.2.4. Espaces

Mettez des espaces autour de tous les opérateurs binaires infixes (`=`, `+`, `*`, `==`, `&&`, `<-`, `%*%`, etc.).

```
# GOOD
x == y
z <- 2 + 1

# BAD
x==y
z<-2+1
```

Mettez des espaces autour de `=` dans les appels de fonction.

```
# GOOD
mean(x = c(1, 2, 3), na.rm = TRUE)

# BAD
mean(x=c(1, 2, NA), na.rm=TRUE)
```

Ne placez pas d'espace pour obtenir des sous-ensembles (`$` et `@`), pour la manipulation des espaces de noms (`::` et `:::`) et pour la génération de séquences (`:`).

```
# GOOD
car$cyl
dplyr::select()
1:10

# BAD
car $cyl
dplyr:: select()
1: 10
```

Mettez un espace après une virgule.

```
# GOOD
mtcars[1, ]
mean(x = c(1, NA, 2), na.rm = TRUE)

# BAD
mtcars[1 ,]
mean(x = c(1,NA,2),na.rm = TRUE)
```

Utilisez un espace avant les parenthèses ouvrantes, sauf dans un appel de fonction.

```
# GOOD
for (element in element_list)
if (total == 5)
sum(1:10)

# BAD
for(element in element_list)
if(total == 5)
sum (1:10)
```

Ne mettez pas d'espaces autour du code entre parenthèses ou entre crochets.

```
# GOOD
if (is_true) message("Hello!")
species["tiger", ]

# BAD
if ( is_true ) message("Hello!")
species[ "tiger" ,]
```

B.2.5. Accolade

Une accolade ouvrante ne doit jamais être placée sur sa propre ligne et doit toujours être suivie d'une nouvelle ligne.

```
# GOOD
if (is_true) {
  # do something
}

if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true)
{
  # do something
}

if (is_true) { # do something }
else { # do something else }
```

Une accolade fermante doit toujours être placée sur sa propre ligne, sauf si elle est suivie de `else`.

```
# GOOD
if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true) {
  # do something
}
else {
  # do something else
}
```

Indentez toujours le code à l'intérieur des accolades.

```
# GOOD
if (is_true) {
  # do something
  # and then something else
}

# BAD
if (is_true) {
  # do something
  # and then something else
}
```

Les accolades et les nouvelles lignes peuvent être évitées, si une déclaration après `if` est très courte.

```
# OK
if (is_true) return(value)
```

B.2.6. Indentation

N'utilisez pas de tabulations ou de mélanges de tabulations et d'espaces pour l'indentation.

Utilisez deux espaces pour l'indentation.

B.2.7. Nouvelle ligne

Dans une définition ou un appel de fonction, les arguments excessifs doivent être indentés sur la ligne de la parenthèse fermante, si deux lignes suffisent.

```
# GOOD
long_function_name <- function(arg1, arg2, arg3, arg4,
                               long_argument_name1 = TRUE)

plot(table(rpois(100, 5)), type = "h", col = "red", lwd = 10,
     main = "rpois(100, lambda = 5)")
```

Sinon, chaque argument peut aller sur une ligne séparée, en commençant par une nouvelle ligne après la parenthèse ouvrante.

```r
# GOOD
long_function_name <- function(long_argument_name1 = c("value1", "value2"),
                               long_argument_name2 = TRUE,
                               long_argument_name3 = NULL,
                               long_argument_name4 = FALSE)

list(
  mean = mean(x),
  sd = sd(x),
  var = var(x),
  min = min(x),
  max = max(x),
  median = median(x)
)
```

Si la condition dans l'instruction `if` s'étend sur plusieurs lignes, alors chaque condition doit se terminer par un opérateur logique, et non commencer par lui.

```r
# GOOD
if (some_very_long_name_1 == 1 &&
    some_very_long_name_2 == 1 ||
    some_very_long_name_3 %in% some_very_long_name_4)

# BAD
if (some_very_long_name_1 == 1
    && some_very_long_name_2 == 1
    || some_very_long_name_3 %in% some_very_long_name_4)
```

Si l'instruction qui contient des opérateurs s'étend sur plusieurs lignes, chaque ligne doit se terminer par un opérateur et non commencer par lui.

```r
# GOOD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2) *
  exp(-(x - d_mean) ^ 2 / 2 / s ^ 2)

# BAD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2)
* exp(-(x - d_mean) ^ 2 / 2 / d_sigma ^ 2)
```

Chaque appel de `dplyr` (après `%>%` ou `|>`) ou de `ggplot2` (après `+`) doit commencer sur une nouvelle ligne.

B.3. Commentaires

Il est recommandé d'écrire les commentaires en anglais.

Les commentaires commencent par `#` suivi d'un espace et du texte du commentaire.

```
# This is a comment.
```

Les commentaires doivent expliquer le pourquoi, et non le comment. Les commentaires doivent expliquer l'intention générale de la commande.

```
# GOOD
# define iterator
i <- 1

# BAD
# set i to 1
i <- 1
```

De courts commentaires peuvent être placés sur la même ligne que le code code.

```
plot(price, weight) # Plot a scatter chart of price and weight
```

Il peut être utile de diviser le code source en morceaux logiques par un commentaire suivi de `-` ou `=`.

```
# Read data ------------------------------------------------------------

# Clean data -----------------------------------------------------------
```

Les descriptions des fonctions et des objets doivent respecter les directives de [*roxygen2*] (https://roxygen2.r-lib.org/).

```
#' Add Together Two Numbers
#'
#' @param x A number.
#' @param y A number.
#' @return The sum of x and y.
#' @examples
#' add(1, 1)
#' add(10, 1)
add <- function(x, y) {
  # general comment
  x + y # inline comment
}
```

Bibliographie

Adler, J. (2010). *R in a nutshell*. O'Reilly.

Allaire, J., Horner, J., Xie, Y., Marti, V., & Porte, N. (2019). *markdown : Render Markdown with the C Library Sundown* [R package version 1.1]. https://github.com/rstudio/markdown

Allaire, J., Xie, Y., McPherson, J., Luraschi, J., Ushey, K., Atkins, A., Wickham, H., Cheng, J., Chang, W., & Iannone, R. (2021). *rmarkdown : Dynamic Documents for R* [R package version 2.11]. https://CRAN.R-project.org/package=rmarkdown

Chang, W. (2012). *R graphics cookbook*. O'Reilly Media.

Chang, W. (2019). *webshot : Take Screenshots of Web Pages* [R package version 0.5.2]. https://github.com/wch/webshot/

Colman, S. M., Pierce, K. L., & Birkeland, P. W. (1987). Suggested Terminology for Quaternary Dating Methods. *Quaternary Research*, *28*(2), 314-319. https://doi.org/10.1016/0033-5894(87)90070-6

Csárdi, G., Müller, K., & Hester, J. (2021). *desc : Manipulate DESCRIPTION Files* [R package version 1.4.0]. https://github.com/r-lib/desc#readme

Dean, J. S. (1978). Independent Dating in Archaeological Analysis. In *Advances in Archaeological Method and Theory* (p. 223-255). Elsevier. https://doi.org/10.1016/B978-0-12-003101-6.50013-5

Frerebeau, N. (2021). *folio : Datasets for Teaching Archaeology and Paleontology* [R package version 1.0.0]. https://CRAN.R-project.org/package=folio

Gillespie, C. (2016). *Efficient R programming : A practical guide to smarter programming*. O'Reilly Media.

Iannone, R. (2020). *DiagrammeR : Graph/Network Visualization* [R package version 1.0.6.1]. https://github.com/rich-iannone/DiagrammeR

Long, J. D. (2019). *R cookbook : Proven recipes for data analysis, statistics, and graphics*. O'Reilly Media.

Lovelace, R. (2020). *Geocomputation with r*. CRC Press.

Murrell, P. (2019). *R graphics*. CRC Press, Taylor & Francis Group.

O'Brien, M. J., & Lyman, R. L. (2002). *Seriation, Stratigraphy, and Index Fossils : The Backbone of Archaeological Dating*. Springer.

R Core Team. (2021). *R : A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/

Schmidt, S. C., & Marwick, B. (2020). Tool-Driven Revolutions in Archaeological Science. *Journal of Computer Applications in Archaeology*, *3*(1), 18-32. https://doi.org/10.5334/jcaa.29

Sievert, C., & Cheng, J. (2021). *bslib : Custom Bootstrap 'Sass' Themes for shiny and rmarkdown* [R package version 0.3.1]. https://CRAN.R-project.org/package=bslib

Silge, J. (2017). *Text mining with R : A tidy approach*. O'Reilly Media.

Wickham, H. (2015). *R packages*. O'Reilly Media.

Wickham, H. (2019). *Advanced r*. Taylor & Francis, CRC Press.

Wickham, H. (2021). *downlit : Syntax Highlighting and Automatic Linking* [R package version 0.4.0]. https://CRAN.R-project.org/package=downlit

Wickham, H., Chang, W., Flight, R., Müller, K., & Hester, J. (2021). *sessioninfo : R Session Information* [R package version 1.2.1]. https://CRAN.R-project.org/package=sessioninfo

Wickham, H., Hester, J., & Ooms, J. (2020). *xml2 : Parse XML* [R package version 1.3.2]. https://CRAN.R-project.org/package=xml2

Wilke, C. O. (2019). *Fundamentals of data visualization : A primer on making informative and compelling figures*. O'Reilly Media, Inc.

Xie, Y. (2016). *bookdown : Authoring Books and Technical Documents with R Markdown* [ISBN 978-1138700109]. Chapman ; Hall/CRC. https://bookdown.org/yihui/bookdown

Xie, Y. (2021a). *bookdown : Authoring Books and Technical Documents with R Markdown* [R package version 0.24]. https://CRAN.R-project.org/package=bookdown

Xie, Y. (2021b). *knitr : A General-Purpose Package for Dynamic Report Generation in R* [R package version 1.36]. https://yihui.org/knitr/

Xie, Y. (2021c). *mime : Map Filenames to MIME Types* [R package version 0.12]. https://github.com/yihui/mime